

SYSTEM DESIGN — HLD NOTES

VIDEO 5 — PROXY & REVERSE PROXY

1. What is a Proxy?

A **Proxy Server** is an intermediate server between the **client and the internet/server**.

Instead of the client directly communicating with the destination:

Client —————→ Internet/Server

the request goes through a proxy:

Client
↓
Proxy
↓
Internet
↓
Server

The proxy acts **on behalf of the client**.

2. Forward Proxy

A normal proxy is generally called a **Forward Proxy**.

Flow

Forward Proxy
↓
Client —————→ Proxy —————→ Internet —————→ Server
↑
acts for client

The server sees the **proxy's IP** instead of directly seeing the client's IP, depending on the proxy configuration/protocol.

Example

Suppose your computer has:

Client IP = 192.168.0.10

Proxy IP = 172.20.10.1

Flow:

192.168.0.10

↓

Proxy

↓

Internet

↓

Server

The proxy represents the client when accessing external resources.

3. Why Use a Forward Proxy?

3.1 IP Privacy / Anonymity

The destination may see the proxy's IP rather than the client's direct IP.

Client

|

| Client IP

↓

Proxy

|

| Proxy IP

↓

Internet

Use case:

Organizations may route outgoing traffic through controlled proxies.

⚠ A proxy does **not automatically provide complete anonymity or privacy**. The proxy itself can potentially see/log traffic depending on the protocol and configuration.

3.2 Access Control

A company can control what employees can access.

Employee



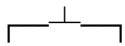
Proxy



Check policy



Allowed?



Yes No



Site Block

Example

A college/company might block:

Gaming websites

Social media

Malicious websites

using a proxy/filtering system.

3.3 Restricted Internet Access

A proxy can enforce network policies and sometimes provide access to resources through a controlled network path.

3.4 Caching

A forward proxy can cache frequently requested resources.

Example:

Client A → Proxy → Website



Cache

Client B → Proxy



Cache Hit



Response

Instead of downloading the same resource from the internet repeatedly, the proxy may serve its cached copy.

Use case

Useful when many users in the same organization request the same content.

3.5 Group/Organization-Level Control

A company can put all outbound traffic through one proxy:

Employee 1 ↘

Employee 2 ↘

Employee 3 ↘ → Forward Proxy → Internet

Employee 4 ↘

Employee 5 ↘

Now the organization gets a central place for:

- Access control
- Monitoring
- Filtering
- Logging
- Caching

4. Reverse Proxy

Now the direction changes.

A **Reverse Proxy** sits in front of **backend servers**.

It acts on behalf of the **server/backend**.

Internet



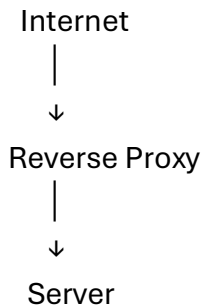
Reverse Proxy

↓

Backend Server

The client doesn't directly communicate with the backend server.

5. Reverse Proxy Flow



The client thinks it is communicating with the public-facing system.

The reverse proxy decides what happens internally.

6. Why Use a Reverse Proxy?

This is extremely important for HLD.

6.1 Hide Backend Servers

Suppose your backend has:

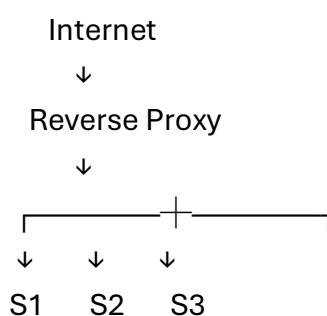
Server 1 → 10.0.0.1

Server 2 → 10.0.0.2

Server 3 → 10.0.0.3

You don't want users directly accessing these servers.

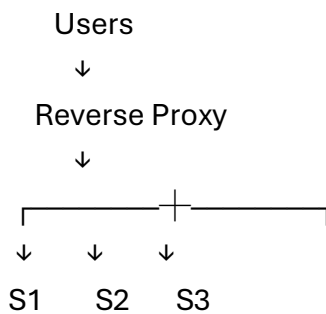
Instead:



Users only interact with the reverse proxy/public endpoint.

6.2 Load Balancing ★

A reverse proxy can distribute traffic across multiple backend servers **if it has load-balancing functionality**.



Example:

Request 1 → S1

Request 2 → S2

Request 3 → S3

Request 4 → S1

This is why your notes connect:

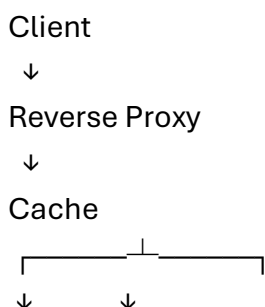
Reverse Proxy → LB

But remember:

Reverse proxy and load balancer are different concepts, although one system can perform both roles.

6.3 Caching

The reverse proxy can cache backend responses.

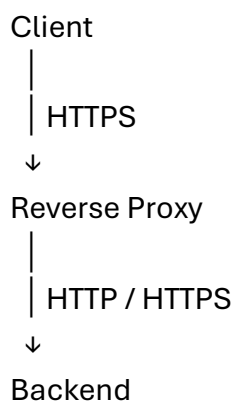


Hit	Miss
↓	↓
Response	Backend

This reduces backend workload.

6.4 SSL/TLS Termination

The reverse proxy can handle HTTPS encryption.



This is called:

TLS termination

The backend doesn't necessarily need to handle the public TLS connection itself.

6.5 Security

A reverse proxy can provide a security boundary in front of backend servers.

It can be integrated with or provide functions such as:

- Request filtering
- IP restrictions
- WAF
- Rate limiting
- Authentication
- DDoS protection mechanisms

You'll learn many of these concepts separately later.

6.6 Routing

A reverse proxy can route requests based on:

URL
Host
Headers

Example:

example.com/users

↓

Reverse Proxy

↓

User Service

example.com/orders

↓

Reverse Proxy

↓

Order Service

7. Forward Proxy vs Reverse Proxy ★ ★ ★

This is an important interview question.

Forward Proxy

Represents the **client**
Used for outgoing traffic
Client knows/uses proxy
Hides/protects client-side identity/network
Access control
Filtering
Client-side caching
Organization/user focused

Reverse Proxy

Represents the **server/backend**
Used for incoming traffic
Client may not know backend structure
Hides/protects backend servers
Load balancing
Caching
Server-side caching
Application/infrastructure focused

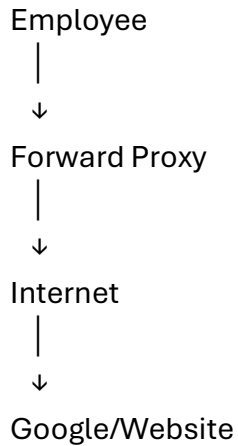
Easy memory trick:

Forward Proxy → Forwarding client's request

Reverse Proxy → Protecting/reaching backend

8. Forward Proxy Example

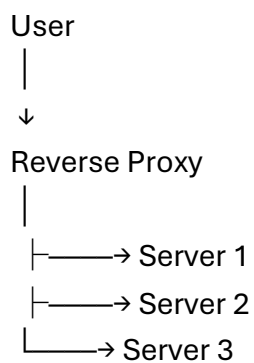
FORWARD PROXY



The proxy is working **for the employee/client**.

9. Reverse Proxy Example

REVERSE PROXY



The proxy is working **in front of the backend**.

10. Proxy vs VPN

Your notes mention this as an interview question.

Both can change the apparent source IP for certain traffic, but they are **not the same thing**.

Proxy

Usually operates at a particular application/protocol level.

Application



Proxy



Internet

A proxy may handle HTTP/HTTPS traffic, for example.

VPN

Creates an encrypted tunnel between the device/network and a VPN endpoint.

Device



Encrypted VPN Tunnel



VPN Server



Internet

A VPN typically routes a broader set of the device's network traffic through the tunnel.

Simple comparison

Proxy	VPN
Often application-specific	Usually network-level
Can be used for HTTP requests, etc.	Can route broader device traffic
Configuration can be per application	Usually configured at OS/network level
May or may not encrypt traffic depending on protocol	VPN tunnel is designed to provide encryption

Remember:

Proxy ≠ VPN

A proxy should not automatically be assumed to provide VPN-level encryption/privacy.

11. Proxy vs Load Balancer

Another important interview question.

Load Balancer

Main purpose:

Distribute traffic among multiple servers.

Users

↓

Load Balancer

└─→ S1

└─→ S2

└─→ S3

Proxy

Main purpose:

Act as an intermediary between two parties.

A proxy may perform many additional functions.

Important point

A reverse proxy **can also perform load balancing.**

So:

Reverse Proxy

+

Load Balancing

is perfectly possible.

But:

Reverse Proxy = Load Balancer

is **not always true.**

12. Can Every Proxy Do Load Balancing?

No.

A proxy is an intermediary.

Load balancing is a specific capability that distributes traffic among multiple backend instances.

Some reverse proxies support load balancing:

Client

↓

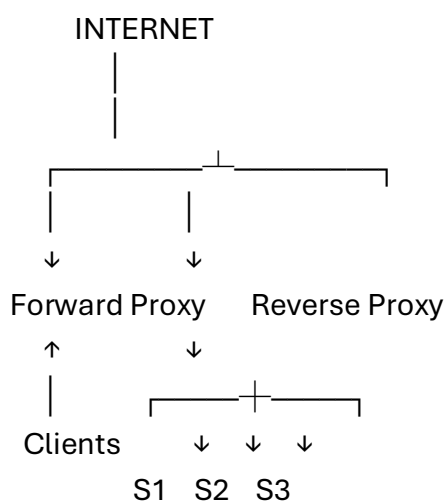
Reverse Proxy + LB

↓

S1 / S2 / S3

But a simple proxy doesn't necessarily provide load balancing.

13. Complete Proxy Architecture



Forward proxy:

Client → Proxy → Internet

Reverse proxy:

Internet → Reverse Proxy → Servers

This two-line diagram is worth remembering for revision.

VIDEO 6 — NETWORKING PROTOCOLS

Now we move from proxy concepts to **network communication fundamentals**.

14. What is a Network Protocol?

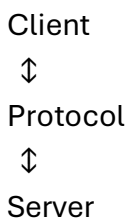
A **network protocol** is a set of rules that defines how devices communicate and exchange data.

It defines things such as:

- How data is formatted
- How data is sent
- How data is received
- How errors are handled
- How devices identify each other
- How communication is established

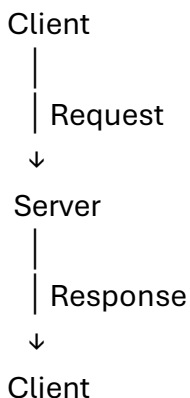
Simple example

When your browser communicates with a web server, both sides need to follow agreed rules.



15. Client-Server Architecture

In the **client-server model**, one side requests a service and another side provides it.



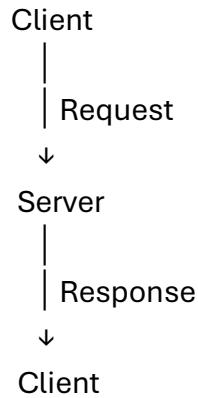
Example

When you open:

youtube.com

your browser acts as the client and communicates with servers.

16. Client-Server Flow



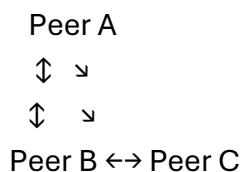
Examples

- Browser → Web server
- Mobile app → API server
- React frontend → Node.js backend
- Email client → Mail server

17. P2P — Peer-to-Peer

In **Peer-to-Peer (P2P)** architecture, devices can communicate directly with each other.

There isn't necessarily one central server handling all communication.



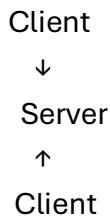
Each peer can act as both:

- Client
- Server

depending on the interaction.

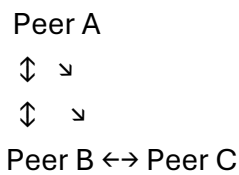
18. Client-Server vs P2P

Client-Server



Centralized.

P2P



More decentralized.

Client-Server	P2P
Central server	Peers communicate directly
Centralized	More decentralized
Easier management	More complex coordination
Server can become bottleneck	Load can be distributed among peers
Common for web applications	Used in certain real-time/file-sharing systems

19. P2P Use Cases

Examples include:

- File sharing
- Certain real-time communication systems
- WebRTC-based communication
- Blockchain networks
- Some distributed systems

Important:

P2P does **not** necessarily mean there is absolutely no server involved.

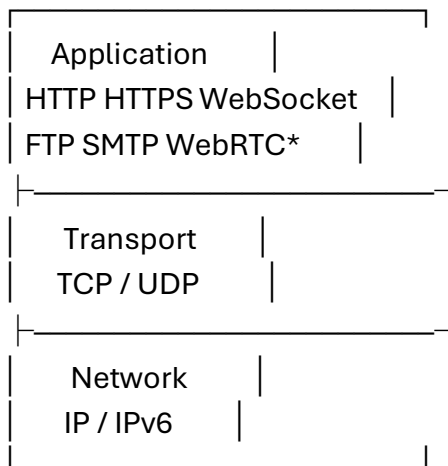
For example, WebRTC applications often use servers for **signaling**, and STUN/TURN infrastructure can assist with connectivity.

20. Where Do TCP and UDP Fit?

This is an important concept.

Network communication is commonly explained using layers.

For your current HLD learning, focus on these three:



* WebRTC is a communication technology/architecture that uses multiple protocols; don't think of it as simply "one application-layer protocol."

21. Network Layer

The **Network Layer** is responsible for getting packets between networks.

A major protocol here is:

IP

Internet Protocol.

Examples:

IPv4

IPv6

Conceptually:

Source



Network Layer



Destination

Think:

Network Layer = Where should the packet go?

22. Transport Layer

Transport layer provides communication between applications/processes running on hosts.

The two major protocols are:

TCP

UDP

Think:

Transport Layer = How should data be delivered between endpoints?

23. Application Layer

This is where application-level communication protocols live.

Examples:

HTTP

HTTPS

FTP

SMTP

WebSocket

Think:

Application Layer = What kind of communication does the application want?

24. TCP — Transmission Control Protocol

TCP provides **reliable, ordered, connection-oriented byte-stream communication**.

Your notes correctly mention:

- Reliable
- Ordered
- Every piece of data should be delivered

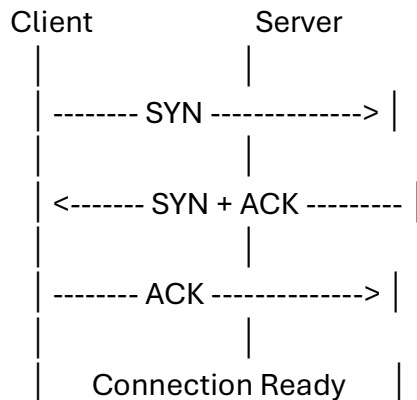
More precisely:

TCP provides mechanisms for reliable, ordered delivery of a byte stream between endpoints.

25. TCP Connection

TCP is **connection-oriented**.

Before normal data transfer, the endpoints establish a connection using the **TCP three-way handshake**.



Then data transfer can begin.

26. Why Does TCP Need a Handshake?

The handshake allows both sides to establish that they can communicate and synchronize connection state.

Simple understanding:

Client → "Can we communicate?"

Server → "Yes, I received you."

Client → "Okay, let's start."

27. TCP Reliability

TCP uses mechanisms such as:

- Sequence numbers
- Acknowledgements
- Retransmission
- Flow control
- Congestion control

So if data is lost:

Packet 1 

Packet 2 

Packet 3 

TCP can detect the missing data and retransmit as needed.

28. TCP Maintains Order

Suppose packets arrive:

3

1

2

TCP can reorder them for the application so the byte stream is delivered in the correct order.

Conceptually:

Network:

3 → 1 → 2

TCP:

1 → 2 → 3

Application:

1 → 2 → 3

29. TCP Use Cases

TCP is useful when **correctness and reliable delivery are more important than minimum latency**.

Examples:

Web traffic

Browser

↓

HTTP/HTTPS

↓

TCP

Traditional HTTP/1.1 and HTTP/2 commonly use TCP.

Other examples

- File transfers
- SSH
- Database connections
- Email protocols such as SMTP/IMAP/POP3

30. UDP — User Datagram Protocol

UDP is:

- Connectionless
- Lightweight
- No built-in guarantee of delivery
- No built-in guarantee of ordering
- No retransmission mechanism like TCP
- Low overhead

Your note says:

Faster than TCP

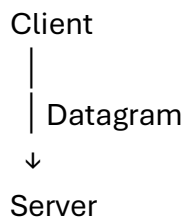
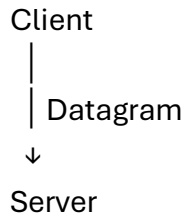
Better way to remember it:

UDP has lower protocol overhead and does not perform TCP-style reliability mechanisms, which can make it suitable for latency-sensitive applications.

Don't memorize simply as "UDP is always faster."

31. UDP Flow

Unlike TCP, there is no TCP-style connection handshake.



The sender can send datagrams without first establishing a TCP-style connection.

32. UDP Doesn't Guarantee Delivery

Suppose:

Packet 1 → 

Packet 2 → 

Packet 3 → 

UDP itself doesn't guarantee that Packet 2 will be retransmitted.

Similarly, packets may arrive out of order.

Sent:

1 → 2 → 3

Received:

1 → 3 → 2

UDP doesn't automatically reorder them for the application.

33. UDP Use Cases

UDP is useful when **low latency is more important than guaranteed delivery**, or when the application implements its own reliability/ordering mechanisms.

Examples:

Real-time voice/video

User A



UDP-based transport



User B

If one video frame is lost, waiting too long for retransmission may be worse than continuing with the next frame.

Online gaming

Player



Game Server

Very old position data may become useless quickly.

DNS

DNS commonly uses UDP for ordinary queries, although TCP is also used in various situations.

WebRTC

WebRTC uses UDP-based transports extensively for real-time media, with additional protocols/mechanisms layered around it.

34. TCP vs UDP ★ ★ ★

This is a **must-revise table**.

TCP	UDP
Connection-oriented	Connectionless
Reliable delivery	No built-in delivery guarantee
Ordered byte stream	Datagram-based; no built-in ordering

Retransmission	No TCP-style retransmission
Flow control	No TCP-style flow control
Congestion control	No TCP-style congestion control
More overhead	Lower overhead
Good for reliable data transfer	Good for latency-sensitive traffic
HTTP/1.1, HTTP/2, SSH, etc.	DNS, real-time media, gaming, WebRTC-related traffic

35. Very Important: HTTP and TCP

The relationship is:

Application

↓

HTTP

↓

TCP

↓

IP

↓

Network

HTTP defines **application-level communication**.

TCP provides **transport-level communication**.

So:

HTTP and TCP are not competitors.

They operate at different layers.

36. HTTP

HTTP = HyperText Transfer Protocol

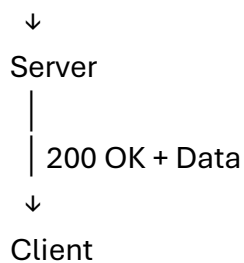
It is an application-layer protocol used for communication between clients and servers.

Example:

Client

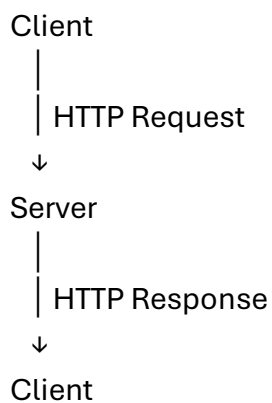
|

| GET /users



37. HTTP Request/Response

Typical flow:



Example:

GET /products

Response:

```
200 OK
[
  ...
]
```

38. HTTPS

HTTPS is essentially:

HTTP secured using TLS

Conceptually:

HTTP
+
TLS

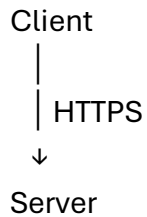
↓

HTTPS

TLS provides security properties such as:

- Encryption
- Integrity
- Server authentication

Flow



The communication is protected by TLS.

Important correction from your notes

Don't write:

HTTPS = HTTP + SSL

Modern HTTPS uses **TLS**. SSL is the older predecessor that is no longer considered secure for modern use.

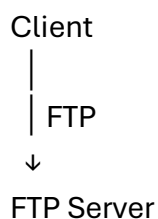
For revision, write:

HTTPS = HTTP over TLS

39. FTP

FTP = File Transfer Protocol

Used for transferring files between systems.



Use case

Uploading/downloading files from an FTP server.

40. SMTP

SMTP = Simple Mail Transfer Protocol

Used primarily for **sending/relaying email**.

Email Client/Server



SMTP



Mail Server

For receiving mail, protocols such as IMAP or POP3 are commonly used.

Easy memory:

SMTP → Send mail

IMAP/POP3 → Retrieve mail

41. WebSocket ★

WebSocket provides a **persistent, bidirectional communication channel** between client and server.

This is different from the traditional request/response style of HTTP.

HTTP

Client → Request → Server

Client ← Response ← Server

Usually the client initiates each request.

WebSocket

Client ←————→ Server
connection
remains open

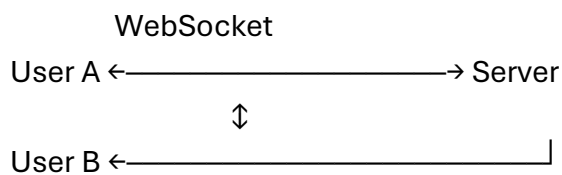
Both sides can send messages independently after the connection is established.

42. WebSocket Use Cases

Very useful for real-time applications:

- Chat applications
- Live notifications
- Live dashboards
- Multiplayer features
- Real-time collaboration

Example:



If a message arrives:

Server
↓
Push message
↓
User

The client doesn't have to repeatedly ask:

"Do I have a new message?"

43. WebRTC

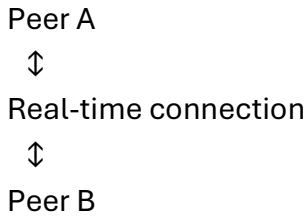
WebRTC = Web Real-Time Communication

It enables real-time communication such as:

- Audio
- Video
- Data

between browsers/devices.

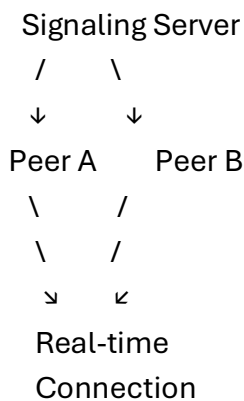
Conceptually:



This is commonly associated with **peer-to-peer communication**.

44. WebRTC Basic Architecture

A simplified architecture:



The signaling server helps peers exchange information needed to establish a connection.

For difficult network situations, infrastructure such as **STUN/TURN** may be involved.

45. WebSocket vs WebRTC

WebSocket	WebRTC
Client ↔ Server	Commonly Peer ↔ Peer
Persistent connection	Real-time peer communication
Server is normally in the communication path	Media/data may flow directly between peers
Great for chat/notifications	Great for audio/video/data

Web application real-time updates

Video calls, voice calls, P2P data

Example

Chat application:

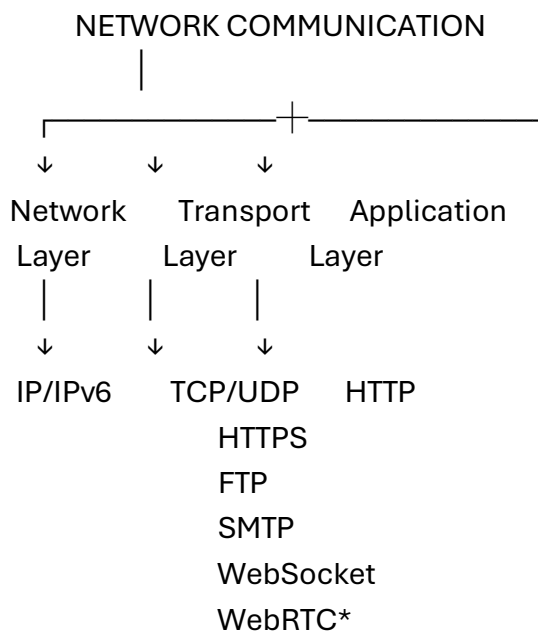
User → WebSocket → Server → WebSocket → User

Video call:

User A ←—— WebRTC ——→ User B

46. COMPLETE PROTOCOL MAP ★ ★ ★

Keep this in your notebook:



* WebRTC is a broader real-time communication technology that uses multiple protocols.

47. Easy Way to Remember Layers

Think about sending a parcel.

Network Layer

Where should it go?

IP addresses help identify source/destination.

Transport Layer

How should communication be delivered between endpoints?

TCP or UDP.

Application Layer

What does the application want to communicate?

HTTP, SMTP, FTP, WebSocket, etc.

So:

Application → WHAT?

Transport → HOW?

Network → WHERE?

This is a very useful mental model.

48. COMPLETE WEB REQUEST FLOW

Suppose you open:

<https://example.com>

A simplified view is:

Browser

|

| HTTPS

↓

HTTP

|

↓

TLS

|

↓

TCP

|

↓

IP

|

↓

Internet

|
↓
Server

The exact stack varies with the protocol/version—for example, HTTP/3 uses QUIC over UDP rather than TCP—but for your current playlist, this simplified HTTP-over-TLS-over-TCP model is important.

49. PROXY + NETWORK PROTOCOLS TOGETHER

Now let's connect Video #5 and #6.

Suppose:

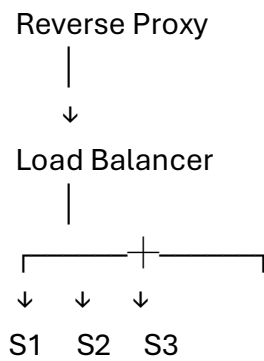
Browser
↓
Forward Proxy
↓
Internet
↓
Reverse Proxy
↓
Load Balancer
↓
Application Server

Communication might conceptually involve:

Application → HTTP/HTTPS
Transport → TCP/UDP
Network → IP

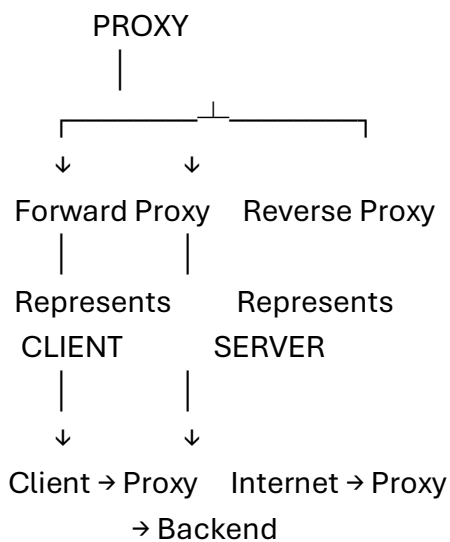
So your HLD understanding is becoming:

CLIENT
|
↓
Forward Proxy
|
↓
Internet
|
↓



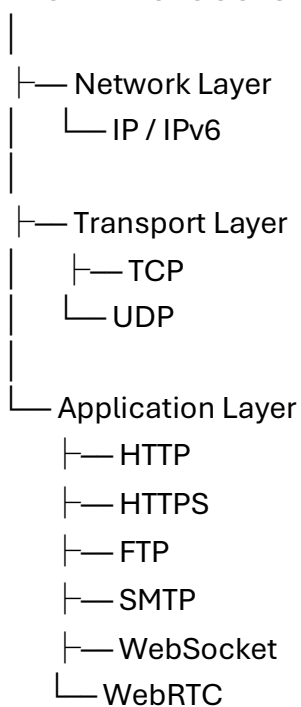
MOST IMPORTANT REVISION FLOW

When you revise Videos 5 & 6, start with this:



Then:

NETWORK PROTOCOLS



INTERVIEW QUESTIONS YOU SHOULD NOW BE ABLE TO ANSWER

Proxy

1. What is a proxy?
2. What is a forward proxy?
3. What is a reverse proxy?
4. Forward proxy vs reverse proxy?
5. Why do we use a reverse proxy?
6. Can a reverse proxy perform load balancing?
7. Is every proxy a load balancer?
8. Proxy vs VPN?
9. Why hide backend servers?
10. How can a reverse proxy perform SSL/TLS termination?

Networking

11. What is a network protocol?
12. Client-server vs P2P?
13. What is TCP?
14. What is UDP?
15. TCP vs UDP?
16. Why does TCP use a three-way handshake?
17. Why might UDP be preferred for real-time communication?
18. What is HTTP?
19. What is HTTPS?
20. HTTP vs HTTPS?
21. What is WebSocket?
22. WebSocket vs HTTP?
23. What is WebRTC?
24. WebSocket vs WebRTC?
25. What are network, transport and application layers?

30-SECOND REVISION

If you have only 30 seconds before an interview:

Forward Proxy: Client-side intermediary. Acts on behalf of clients.

Reverse Proxy: Server-side intermediary. Sits in front of backend servers.

Reverse Proxy uses: Load balancing, caching, TLS termination, routing, security, hiding backend infrastructure.

Load Balancer: Distributes traffic among server instances.

TCP: Reliable, ordered, connection-oriented byte stream.

UDP: Connectionless datagrams with no built-in guarantee of delivery or ordering.

HTTP: Application-layer request/response protocol.

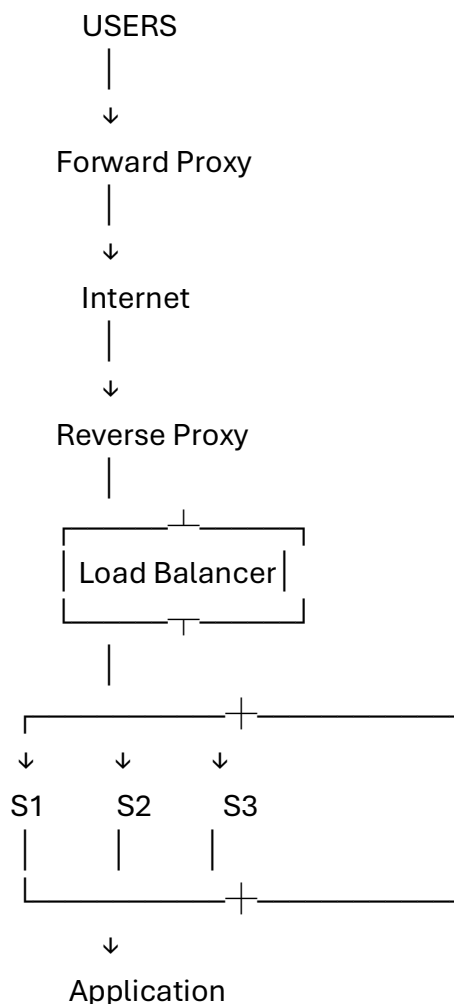
HTTPS: HTTP over TLS.

WebSocket: Persistent bidirectional client-server communication.

WebRTC: Real-time audio/video/data communication, commonly peer-to-peer.

P2P: Peers can communicate directly and act as both clients and servers depending on the interaction.

One final architecture to memorize





And underneath the communication:

Application Layer



HTTP / HTTPS / WebSocket / SMTP / FTP



Transport Layer



TCP / UDP



Network Layer



IP / IPv6

This is the level of understanding you should have before moving to Video #7/#8. You don't need to memorize every networking detail yet—the later videos will build on these concepts, especially **scaling, caching, distributed systems, CDN, rate limiting, and consistent hashing**.